Philipps-University Marburg
Department Mathematics and Computer Science
Working Group Database Systems

Wintersemester 2025
05.12.2025

Prof. Dr. Bernhard Seeger
Manuel Oed

Exercises for the lecture
NoSQL-Database Systems

Submission: 19. 12. 2025,
no later than 10:00 Uhr
via ILIAS

Exercise 4

Note: In the following exercises, you are to practice working with the different APIs of Apache Spark (https://spark.apache.org/). In your solutions, limit yourself as much as possible to the respective API. Furthermore, for these exercises, the **Java API** of Spark should be used. Use the provided code template to automatically include the required libraries via Maven.

Task 4.1: Map Reduce and RDDs (3+3+3+3+3) (15 Points)

In this exercise, you will practice working with Apache Spark's **Resilient Distributed Dataset (RDD)** API and the *MapReduce* principle (see, e.g., here). For the following sub-tasks, use the provided file `resources/words.txt` as input. You need to read it into a `JavaRDD` using Spark. Use the Spark `RDD` API for your solutions, and for each sub-task, use at least one call to `mapToPair`. Complete all tasks in the class `MapReduceSpark`.

a) Create a method `wordCount`. Solve the well-known *WordCount* problem by counting the occurrences of words in the file `words.txt`. Ensure that punctuation marks (, . ? ! ;) are not considered part of the words. Print the first 10 results to the console, sorted in descending order by occurrence.

b) Create a method `longestWords` that prints the 10 longest words to the console (the first 10 words when sorted by length in descending order).

c) Create a method `averageWordLength` that prints the average word length in the text to the console.

d) Create a method `anagramm`. Group the words that are anagrams of each other. Print the groups with more than one anagram to the console.

e) A *bigram* is a pair of consecutive words in a text. Implement a method `consecutiveWords` that counts how many times each bigram appears. Print to the console every bigram that occurs more than 10 times, along with its count.

Task 4.2: Spark DataFrames (2+2+2+3+3+4+4) (20 Points)

In this exercise, you will practice working with the Apache Spark DataFrames Java API (see, e.g., here). Use the Spark DataFrames API (i.e., as in the lecture, using a `Dataset<Row>`) to solve the tasks. As input for the `Dataset<Row>`, use the provided file `resources/meteorite.json`, which contains meteorite impact data recorded by NASA.

a) Read the file `meteorite.json` with Spark and print the schema inferred by Spark.

b) In the class `DataFrameAnalytics`, create a method `averageMass` that prints the average mass of the meteorites to the console.

c) In the class `DataFrameAnalytics`, create a method `fallClassification` that prints, for each distinct fall category, the number of meteorites belonging to that category to the console.

d) In the class `DataFrameAnalytics`, create a method `noCoordinates` that prints meteorite impacts with missing geolocation. Sort these impacts by date in ascending order before printing.

e) In the class `DataFrameAnalytics`, create a method `dataCleaning`. Check whether there are records where the two latitude and longitude values differ. Since these are floating-point numbers, use an epsilon of 0.1 for the comparison. Print the result to the console.

f) In the class `DataFrameAnalytics`, create a method `pointDistance`. Register a UserDefinedFunction in Spark that calculates the distance between two point coordinates using the *Haversine formula*. Then use it to measure the distance between all meteorite impacts. Print the 15 closest impact pairs to the console.

g) Create a class `GeoLocation` with fields `type` (String), `longitude` (double), and `latitude` (double), along with corresponding getter and setter methods. Then, in the class `DataFrameAnalytics`, create a method `addDataPoints`. First, create a new variable `originLocations` of type `Dataset<GeoLocation>`, populated using an `Encoder` with geolocation information from your JSON file. Then create a list of `3` custom `GeoLocation` objects. Convert this list into a `Dataset<GeoLocation>` using an `Encoder` and store it in `addedLocations`. Finally, union `originLocations` and `addedLocations` into a new `Dataset<GeoLocation>` and print the result to the console.

Note: Pay attention to the order of attributes when performing the union. When creating the dataset with the added locations, it might help to select the individual columns and then apply `.as(encoder)`.